

性能对比实验报告

单体与微服务性能对比实验报告

实验日期：2026-08-28

1. 实验结论

本次实验在同一台机器、同一 Docker 引擎、同一批 200 条固定商品数据上，使用同一份 k6 脚本，
分别对单体和微服务的 3 个主要商品接口各运行 3 次。18 个 case 全部通过，错误率均为 0%。

接口	微服务吞吐量变化	微服务平均延迟变化	微服务平均 CPU 变化	微服务平均内存变化
商品列表	+61.9%	-38.6%	+55.2%	+5.1%
商品搜索	+53.9%	-35.5%	+50.7%	+17.3%
商品详情	+30.9%	-26.2%	+6.9%	+14.0%

可以得出的结论是：在本次限定条件下，当前微服务实现的三个商品查询接口吞吐量更高、响应延迟更低，
但服务端总 CPU 和内存开销更高。不能据此写“微服务架构天然更快”。

2. 测试条件

项	条件
机器	Apple M5, 10 核 CPU, 16GB 内存
操作系统	macOS 26.5.1, arm64
容器运行时	Docker Desktop / Docker Engine 29.7.2
压测工具	k6 2.2.0, Darwin arm64
单体入口	http://127.0.0.1:3001
微服务入口	http://127.0.0.1:8081, 经 API Gateway 访问 Product Service
压测模型	closed model, 5 个固定 VU, 每个 case 20 秒, 无思考时间
运行次数	每个版本、每个接口 3 次, 共 18 个 case

项	条件
压测脚本	04_tests/performance/k6-interface-comparison.js，仅通过环境变量切换入口和接口
数据	两库各 200 条 PERF-COMPARE 商品，ID 990001-990200
数据校验	两边业务字段 MD5 均为 660bc7302948c0b2c958e3201e9ed8dd
顺序控制	第 2 轮反转版本先后顺序；三轮轮换接口顺序
CPU/内存采样	每约 2-3 秒执行一次 docker stats --no-stream，保存逐容器原始值

资源统计采用服务端总成本：

- 单体：softw-backend + softw-mysql。
- 微服务：api-gateway + product-service + microservice-mysql。
- CPU 为同一采样时刻各容器 CPU 百分比之和，允许超过 100%；内存为各容器已用内存之和。

3. 测试接口

编号	接口	固定请求
PERF-API01	商品列表	GET /api/products? category=performance&page=1&limit=20&sort=price_asc
PERF-API02	商品搜索	GET /api/products/search?keyword=PERF-COMPARE- 150&category=performance&page=1&limit=20
PERF-API03	商品详情	GET /api/products/990001

所有请求都断言 HTTP 200，并断言响应包含固定数据；详情接口还断言 id=990001。

4. 三轮原始结果

CPU 列为平均/峰值，内存列为平均/峰值。吞吐量单位为 req/s，时间单位为 ms，内存单位为 MiB。

版本	接口	轮次	并发	请求数	吞吐量	平均	P95	错误率	CPU	内存
单体	列表	1	5	20912	1045.380	4.691	6.312	0%	132.448/147.620	544.011/564.100
单体	列表	2	5	19960	997.800	4.927	6.307	0%	134.830/141.450	536.350/556.900

版本	接口	轮次	并发	请求数	吞吐量	平均	P95	错误率	CPU	内存
单体	列表	3	5	20047	1002.143	4.904	6.237	0%	135.665/141.620	542.345/568.100
微服务	列表	1	5	33235	1661.559	2.933	4.225	0%	227.716/280.710	554.996/573.960
微服务	列表	2	5	33947	1697.101	2.870	3.760	0%	183.132/212.320	592.510/598.520
微服务	列表	3	5	31410	1570.308	3.106	4.240	0%	214.664/223.030	558.747/567.080
单体	搜索	1	5	39103	1954.964	2.512	3.345	0%	166.619/180.090	521.615/549.100
单体	搜索	2	5	35263	1762.956	2.794	3.675	0%	170.809/203.770	510.465/525.900
单体	搜索	3	5	36261	1812.833	2.717	3.605	0%	155.208/180.960	521.283/557.600
微服务	搜索	1	5	58783	2939.002	1.664	2.464	0%	243.471/266.220	611.227/634.740
微服务	搜索	2	5	57116	2855.633	1.712	2.511	0%	236.322/264.520	613.220/644.870
微服务	搜索	3	5	54297	2714.705	1.803	2.652	0%	262.450/272.230	597.840/640.590
单体	详情	1	5	22696	1134.539	4.363	5.717	0%	119.217/140.050	510.795/522.800
单体	详情	2	5	18192	753.491	6.582	8.644	0%	140.697/154.560	509.854/534.400
单体	详情	3	5	21322	1065.908	4.645	6.019	0%	129.055/139.630	512.391/538.100
微服务	详情	1	5	26584	1328.998	3.715	4.991	0%	142.834/150.170	601.459/646.540
微服	详情	2	5	25816	1290.611	3.826	5.271	0%	131.751/148.720	595.210/599.320

版本	接口	轮次	并发	请求数	吞吐量	平均	P95	错误率	CPU	内存
务										
微服务	详情	3	5	24957	1247.608	3.959	5.583	0%	141.235/212.960	551.666/556.180

5. 三轮均值

版本	接口	吞吐量	平均响应	P95	错误率	平均 CPU	平均内存
单体	列表	1015.108	4.841	6.285	0%	134.314%	540.902MiB
微服务	列表	1642.989	2.970	4.075	0%	208.504%	568.751MiB
单体	搜索	1843.584	2.674	3.542	0%	164.212%	517.788MiB
微服务	搜索	2836.447	1.726	2.542	0%	247.414%	607.429MiB
单体	详情	984.646	5.197	6.793	0%	129.656%	511.013MiB
微服务	详情	1289.072	3.833	5.282	0%	138.607%	582.778MiB

6. 差异分析

1. 单体商品列表会关联查询 Users 表并组装卖家对象；微服务商品表保存卖家快照，查询路径更聚焦。
在当前固定数据和只读接口上，这部分差异超过了 API Gateway 多一次转发的成本。
2. 微服务版本的 Express 中间件和商品控制器更精简；单体还执行全局安全、限流、请求追踪及较完整的
ORM 关联处理，因此本次延迟更高。它们是当前实现差异，不能直接归因于“拆成微服务”。
3. 微服务需要同时运行网关和商品服务两个 Node.js 进程，且使用独立 MySQL 容器，因此三个接口的
内存均高于单体，列表和搜索的 CPU 也明显更高。这是拆分和额外网络/序列化带来的资源成本。
4. 详情接口两边都会增加浏览次数，包含一次数据库写操作；该接口的微服务 CPU 优势很小，说明涉及
写操作后，两种实现的差距比纯查询更小。

7. 原始证据与复现

```
npm run perf:compare
```

脚本会自动写入同一批数据、校验两边 MD5、执行 18 个 case、生成 CSV，并在退出时清理实验数据。

- 汇总 CSV: 04_tests/reports/performance/interface-comparison-2026-08-28.csv
- 原始目录: 04_tests/reports/performance/raw/interface-comparison-2026-08-28/
- 每个 case 的 .json: k6 机器可读指标。
- 每个 case 的 .txt: k6 原始控制台报告。
- 每个 case 的 -stats.tsv: 逐时间点、逐容器 CPU 和内存。
- dataset-verification.txt: 两边数据数量、ID 范围和 MD5 校验结果。

限制：这是单机 Docker、5 VU、20 秒的短时实验，不代表公网延迟、多节点 Kubernetes 或长期稳定性。

结论只适用于本报告的代码版本、数据规模、接口和运行环境。